

OSSP – Practical 4

Process Synchronization using wait() and waitpid() and Zombie Process Handling

1. Aim

To create multiple child processes and synchronize their completion using wait() and waitpid(), compare their behavior, and create and handle a zombie process using proper synchronization techniques.

2. What We Did

A C program was executed to create three child processes. The child processes displayed their process IDs, while the parent synchronized their completion using wait() and waitpid(). A separate zombie-process demonstration was also executed, where the parent waited for the child before completing.

3. Compilation and Execution

```
$ gcc waitex.c
$ ./a.out
$ gcc Zombie.c
$ ./a.out
```

4. Output – Multiple Child Processes

```
manu@Manasvi:~$ gcc waitex.c
manu@Manasvi:~$ ./a.out
child 2 pid = 3152
child 1 pid = 3151
child 3 pid = 3153
parent: Any one child complete with wait()
parent: child 3 completes with waitpid()
all the childs completed
manu@Manasvi:~$ |
```

Observation: Three child processes were created with PIDs 3152, 3151 and 3153. The parent reported completion using wait(), then specifically reported completion of child 3 using waitpid(), and finally confirmed that all children completed.

5. Output – Zombie Process Handling

```
manu@Manasvi:~$ gcc Zombie.c
manu@Manasvi:~$ ./a.out
parent process 7000
child process 7001
Parent process completed
manu@Manasvi:~$ |
```

Observation: The parent process (PID 7000) and child process (PID 7001) were created successfully. The child completed, and the parent subsequently displayed "Parent process completed" after synchronization.

6. wait() vs waitpid()

Function	Behavior
wait()	Waits for any one child process to terminate.
waitpid()	Waits for a specific child process using its PID.

7. Zombie Process

A zombie process occurs when a child process terminates but the parent has not yet collected its exit status. The process table can be investigated using the ps command. Using wait() or waitpid() allows the parent to collect the child's termination status and removes the zombie entry.

8. Result

Multiple child processes were successfully created and synchronized using wait() and waitpid(). The difference between both functions was observed. A separate child-parent execution was also performed to demonstrate proper synchronization during zombie-process handling.